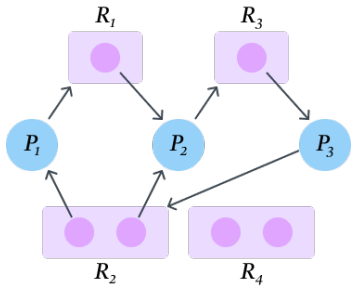
Deadlock-1

A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set.



Resource Allocation Graph with Deadlock

Deadlock Characterization

- **Mutual exclusion:** only one process at a time can use a resource
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait:** a set of processes $\{P_1, P_2, \ldots, P_n\}$ are waiting for each other in a circular chain.

Deadlock Avoidance

- Simplest and most useful model requires that each process declare the maximum number of resources of each type that it may need.
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
- Resource-allocation state is defined by the number of available and allocated resources, and the maximum demands of the processes.

Deadlock Prevention

- **Mutual exclusion:** not required for sharable resources; must hold for non-sharable resources.
- **Hold and wait:** require processes to request and be allocated all resources before execution, or allow processes to request resources only when they have none.
- **No preemption:** if a process holding resources requests a new unavailable resource, all its currently held resources are released and added to the waiting list. The process restarts only when it can reclaim both its old resources as well as the new ones.
- **Circular wait:** impose an ordering of all resource types, and require that each process requests resources in an increasing order of enumeration.

Safe State

- When a process requests an available resource, the system must decide if immediate allocation leaves the system in a **safe state**.

- System is in a **safe state** if there exists a sequence $\langle P_1, P_2, \ldots, P_n \rangle$ of all the processes in the system such that for each P_i, the resources that P_i can still request can be satisfied by the currently available resources + the resources held by all preceding processes P_j, where $j < i$.

  1. If P_i's resource needs are not immediately available, then P_i can wait until all P_j have finished.
  2. When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate.
  3. When P_i terminates, P_{i+1} can obtain its needed resources, and so on.

Deadlock-2

Banker's Algorithm

Let, n : number of processes m : number of resource types

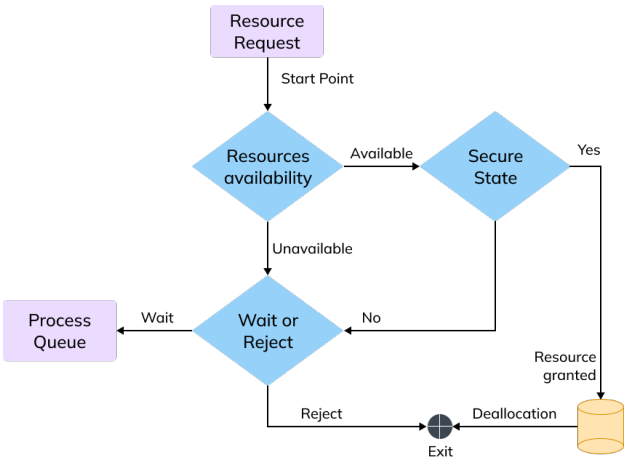
- **Available**: A vector of length m. $Available[j] = k \Rightarrow$ There are k instances of resource type R_j available.

- **Max**: $n \times m$ matrix. $Max[i, j] = k \Rightarrow$ Process P_i may request at most k instances of resource type R_j.

- **Allocation**: $n \times m$ matrix. $Allocation[i, j] = k \Rightarrow$ Process P_i is currently allocated k instances of resource type R_j.

- **Need**: $n \times m$ matrix. $Need[i, j] = k \Rightarrow$ Process P_i may need k more instances of resource type R_j to complete its task.

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$



Safety Algorithm:

1. Initialization:
   - Let *Work* and *Finish* be vectors of length m and n, respectively.
   - *Work* $=$ *Available*
   - *Finish*$[i] =$ false, for $i = 0, 1, \ldots, n-1$
2. Find an i such that both:
   (a) *Finish*$[i]$ = false, and
   (b) *Need*$_i \leq$ *Work*
   If no such i exists, go to step 4.
3. *Work* $=$ *Work* $+$ *Allocation*$_i$
   Finish$[i] =$ true; Go to step 2.
4. If *Finish*$[i]$ == true for all i, then the system is in a safe state.

Resource-Request Algorithm for Process P_i

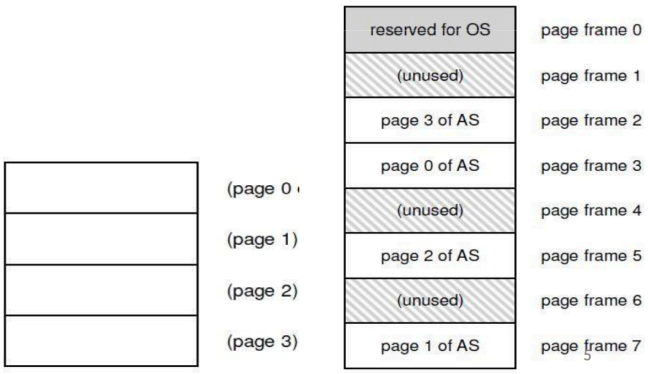
Let $Request_i$ = request vector for process P_i. If $Request_i[j] = k$, then process P_i wants k instances of resource type R_j.

1. If $Request_i \leq Need_i$, go to Step 2.
   Otherwise, raise an error condition, since process P_i has exceeded its maximum claim.
2. If $Request_i \leq Available$, go to Step 3.
   Otherwise, P_i must wait, since the requested resources are not immediately available.
3. Pretend to allocate the requested resources to P_i by modifying the state as follows:
   $Available = Available - Request_i$
   $Allocation_i = Allocation_i + Request_i$
   $Need_i = Need_i - Request_i$
   - If *safe*, $\Rightarrow$ the resources are allocated to P_i.
   - If *unsafe*, $\Rightarrow$ P_i must wait, and the old resource-allocation state is restored.

Virtual Memory

How is actual memory reached?

- Virtual addresses (VA) are translated to physical addresses (PA) to access actual memory.
- CPU works with VA but memory hardware uses PA.
- OS allocates memory and tracks process locations.
- Memory Management Unit (MMU) performs translation using info provided by the OS.



Example: Paging

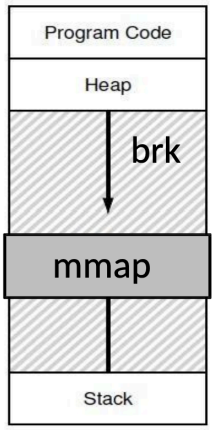
- OS divides VA space into fixed size into **pages** and physical memory into **frames**.
- To allocate memory, a page is mapped to a free frame
- Page table stores mappings: virtual page $\rightarrow$ physical frame for a process.
- MMU uses page table to translate VA $\rightarrow$ PA.

How does the OS allocate memory?

- OS needs memory for its data structures.
- For large allocations, OS allocates a page.
- For smaller allocations, OS uses various memory allocation algorithms.
  - Cannot use libc and malloc in kernel!

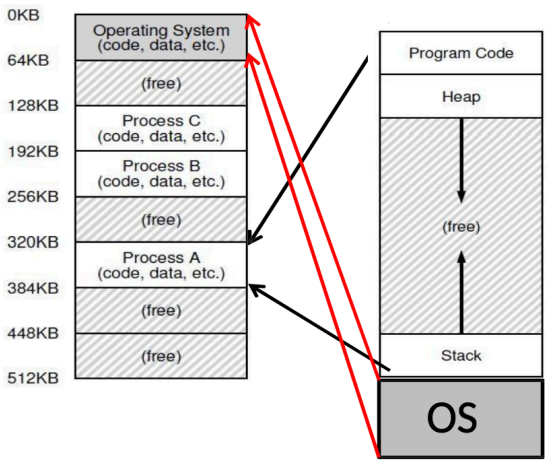
What are the system calls used for memory allocation?

- `malloc` is implemented by the C standard library.
- Uses algorithms for efficient memory allocation and free space management.
- To grow the heap, the C library typically uses the `brk`/`sbrk` system calls.
- Programs can also allocate page-sized memory regions with the `mmap()` system call.
- `mmap()` can request "anonymous" pages directly from the OS.



What is the address space of the OS?

- The OS is not a separate process with its own address space.
- Instead, OS code is part of the address space of every process.
- A process sees the OS as part of its code (similar to a library).
- Page tables map the OS addresses to OS code.



File I/O

Hard links

- Hard linking creates another file that points to the same inode number (and hence, same underlying data)
- If one file deleted, file data can be accessed through the other links
- Inode maintains a link count, file data deleted only when no further links to it
- You can only unlink, OS decides when to delete.

```
prompt> echo hello > file
prompt> cat file
hello
prompt> ln file file2
prompt> cat file2
hello

prompt> ls -i file file2
67158084 file
67158084 file2

prompt> rm file
removed 'file'
prompt> cat file2
hello
```

Soft links

- Soft link is a file that simply stores a pointer to another filename.

```
file
file2 > file
```

- If the main file is deleted, then the link points to an invalid entry: **dangling reference**.

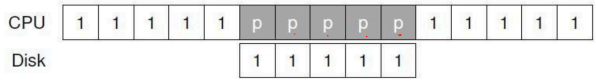
```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
prompt> rm file
prompt> cat file2
cat: file2: No such file or directory
```

What problems do we face while simple execution of an 1/O device? How to solve them? Explain in detail.

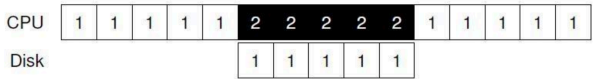
- Polling status to see if device ready- wastes CPU cycles.
- Programmed I/O- CPU explicitly copies data to/from device.

Interrupts

- Polling wastes CPU cycles.
- Instead, OS can put the process to sleep and switch to another process.

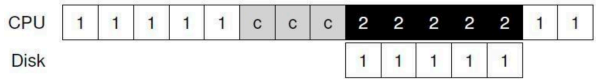


- When an I/O request completes, the device raises an interrupt.

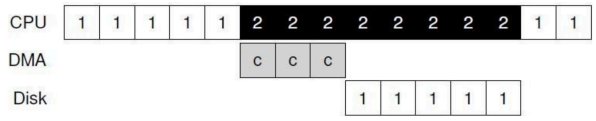


Direct Memory Access (DMA)

- CPU cycles wasted in copying data to/from device



- Instead, a special piece of hardware (DMA engine) copies from main memory to device
  - CPU gives DMA engine the memory location of data
  - In case of read, interrupt raised after DMA completes
  - In case of write, disk starts writing after DMA completes



Paging and Demand Paging

Page algos

Page Fault

- Present bit in page table entry: indicates if a page of a process resides in memory or not
- When translating VA to PA, MMU reads present bit
- If page present in memory, directly accessed
- If page not in memory, MMU raises a trap to the OS - page fault

Translation Lookaside Buffer (TLB)

- A cache of recent VA-PA mappings
- To translate VA to PA, MMU first looks up TLB
- If TLB hit, PA can be directly used
- If TLB miss, then MMU performs additional memory accesses to “walk” page table
- TLB misses are expensive (multiple memory accesses)
  - Locality of reference helps to have high hit rate
- TLB entries may become invalid on context switch and change of page tables

Demand Paging

- Memory management technique where pages are loaded into physical memory only when needed (on first access).
- Reduces memory usage by avoiding loading unused pages.
- If a required page is not in memory, a page fault occurs $\rightarrow$ OS loads the page from disk into a free frame.
- Common in virtual memory systems for efficient resource use.